



JavaRunner - Diseño de proyecto

Traductores

Docente: Guillermo Licea Sandoval

Alumnos:

- Chaparro Herrera Hugo Giovanni - 2200357
- Rivera Vazquez Hugo Alexis - 2200073

Tijuana, Baja California a 22 de mayo de 2026



Tabla de contenido

Fase de análisis léxico.....	3
Token.java.....	3
Definición de constantes.....	3
Creación de variables.....	3
Lexer.java.....	4
Variables declaradas.....	4
Métodos implementados.....	4



Fase de análisis léxico

La fase de análisis léxico representa la primera fase del proyecto. Su función principal es tomar el código fuente de entrada, descartar caracteres como espacios o comentarios y convertir la secuencia de caracteres en una lista de componentes léxicos (tokens) significativos.

Token.java

La clase Token actúa como una estructura de datos inmutable que representa la unidad léxica mínima de información dentro del código fuente. Los componentes principales que conforman la clase son:

Definición de constantes

Se crea un conjunto de constantes la cual contiene palabras reservadas que conforman el lenguaje de programación Java, además de literales e identificadores, operadores y condicionales e indicadores de fin de archivo.

```
Java
public enum Type {
    // Keywords
    INT, DOUBLE, STRING,
    IF, ELSE, WHILE,

    ...

    // Literales / Ids
    IDENTIFICADOR,
    NUMERO_ENTERO,
}
```

Creación de variables

Se declararon variables para el almacenamiento del string extraído del código fuente (data). Además de registro de la línea donde se encontró el token dentro del código (line). Además del constructor y un toString que utilizaremos para imprimir cada Token en un formato específico.

```
Java
public final Type type;
public final String data;
public final int line;

public Token(Type type, String data, int line) {
    this.type = type;
```

```
        this.data = data;  
        this.line = line;  
    }  
  
    @Override  
    public String toString() {  
        return "Token(" + type + ", \"" + data + "\", línea = " + line + ")";  
    }  
}
```

Lexer.java

El aplica la lógica de autómatas para leer el código fuente carácter por carácter y agruparlos en objetos Token.

Variables declaradas

- **input**: Almacena el código fuente dentro de un String.
- **pos**: Almacena el puntero del índice actual de la lectura del código fuente.
- **line**: Contador de la línea actual que se incrementa al detectar saltos de línea.

Java

```
private final String input;  
private int pos = 0;  
private int line = 1;
```

Métodos implementados

- **tokenize()**: Método orquestador. Itera sobre el texto de entrada evaluando los caracteres para delegar la lectura al método especializado correspondiente. Retorna la colección completa de objetos Token.
- **skipWhitespaceAndComments()**: Avanza el puntero ignorando espacios en blanco, tabulaciones y comentarios.
- **readNumber()**: Extrae secuencias numéricas. Pudiendo diferenciar entre enteros y reales.
- **readString()**: Extrae literales de cadena delimitados por comillas dobles.
- **readIdentifierOrKeyword()**: Lee palabras y las compara contra el mapa estático keyword. Si hace match, retorna el token de la palabra reservada; de lo contrario, asume que es un identificador.
- **readOperator()**: Utiliza un lookahead para distinguir entre operadores simples y compuestos.